Place and Sort 3 DoF Robot Manipulator

Ahmed Daw *52-6444* | Nabil Hassan *52-11233* | Amr Mahgoub *52-12977* | Mohamed Yasser *52-12906* | Mohamed Ashraf *52-17791* | Yassin Eissa *52-20855*

***Index Terms*—Pick and Place, Warehouse, Control, Gripper Design, MATLAB, Simscape, CoppeliaSim, Trajectory Planning**

I. Introduction

Robotic manipulators have been extensively used across a wide range of applications, from warehouses, medical service robots, factories, and various other industrial applications, contributing to the global shift towards automation in industrial processes and aligning with the objectives of Industry 4.0, enhancing functionality, efficiency, and scalability, making robot manipulators a cornerstone in the ongoing smart manufacturing revolution.

The field of robotic manipulators is a well-established area of research, particularly in industrial applications. They are extensively employed in the majority of factories and warehouses operated by some of the world's most influential conglomerates. From automotive giants like Toyota to cutting-edge healthcare facilities such as the pharmacist robot at Dubai Hospital.

One of the most common industrial applications of robotic manipulators is the pick & place manipulators, where two manipulators collaborate to move an object from one place to another, whether for further machining or delivery, when used in warehouses, sorting boxes or containers is also a common application, where the 2nd robot is able to arrange received containers according to different criteria, such as color, size or material.

In our application, we aim to sort objects based on color, using either a set of push buttons to let the robot know which trajectory should it take, along the lines of blind sorting, where the robot by itself is unable to detect the colour of the container, but from a user input, it is able to pick a box from a preset point on the path, traveling in a set of pre-defined trajectory, reaching a preset final destination, and placing the container there.

II. Literature Review

The "Pick and Stow (Place)" problem is one of the more classical robotics problems, where researchers have thoroughly investigated it over the last few decades [1], as they are widely employed in most factories and warehouses of the most impactful conglomerates across the globe, as contrary to conveyors and traditional sorting machines, robots have proven to be more efficient in transporting items within a warehouse environment [1].

Despite their effectiveness, there are a set of challenges, mainly in control and trajectory planning. In their research, Prakash et al, 2020, proposed a dual-loop optimal hierarchical control scheme for robotic manipulators, which consists of an outer and an inner loop, where the outer loop provides the joint velocity reference signal to the inner loop, where kinematic control is formulated as a closed loop optimal control to generate trajectories, where the desired target can be determined using the feedback from the actual robot state, and the kinematic control is obtained by a closed-form analytic solution of the Hamilton-Jacobi-Belhlman (HJB) equation [1].

Martinez et al., 2015, have studied the bin picking problem from a computer vision perspective, by integrating a vision system with the robotics system, where that system is used to locate and validate if the located bins are pickable, signalling for the chosen ABB IRB2400 robot to pick the bin and correctly place it in the desired location. The team also showcased the tool design and calibration process, where the final designed tool had one gripper, despite the ability of the vision system to detect multiple bins simultaneously, however, increasing the number of grippers increases the risk of collisions, and moreover, the researchers have designed a compliant tool component to facilitate smooth gripping; this component is made up of a gas filled device that is used to compensate for potential collisions between the tool and the part [2].

Borkar et al., 2017 have designed a pick and place robot to transfer a water container using a gripper end effector, where there is established wireless communication between the mobile robot and the mobile base station, the two most important tasks in their project were programming the AVR on both stations, where they used Atmega16, and the programming of the Graphical User Interface (GUI). The researchers calculated that the maximum weight the arm can carry is 4.95kg, based on the calculations using the DC motor values and the generated torques. A 2-fingered gripper was also used as an end effector, in the case of using soft material finger pads, the grippers can be modeled based on the exerted force, to avoid slippage as follows:

$$\mu n_f F g = w \tag{1}$$

where μ is the coefficient of friction between the work part and the fingers, n_f is the number of fingers contacting, Fg is the gripping force and w is the weight of the grasped object.

If the weight of the work part is greater than the force applied to cause the slippage, equation (1) must be changed by adding a g factor to the weight [3].

Kumar et al, 2017 have designed and developed an automated pick and stow for an e-Commerce Warehouse, where in their paper, they provided details about the main modules to accomplish this feat, from the perception module, to the planning module, the calibration module and the gripping and suction system. The perception module is responsible for recognizing query items and accurately localizing them in the 3-D workspace, the planning module in this paper, focuses on motion planning, which is used when internal access to the motor controllers is not available, motion planning provides suitable joint angle or velocity trajectories required to take the robot from one pose to another. The designed robot is able to avoid obstacles, using path planning algorithms, the pick sequence is primarily involves 4 steps as shown in Fig 1, this sequence is carried out using a set of pre-defined joint angles, since these poses do not vary with time, however, pre grasp motion and post grap motions are carried out using RRT algorithm, as the desired pose required for grasping varies by object, therefore, real time planning is required.

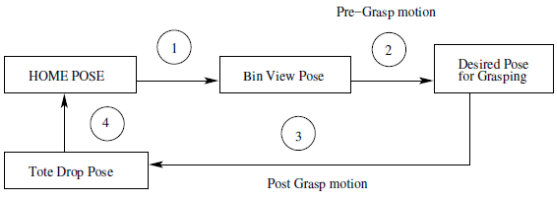

Fig. 1. Sequence of motion planning picking task

For the gripper design, the team combined a parallel jaw gripper with suction system, where the suction was only used when jaw gripping was not possible, the gripper uses a single actuator, which utilizes rack pinion mechanism to achieve linear motion between the fingers. The robot manipulator model was carried by computing the forward kinematics of the system, by deriving the D-H parameters for the UR5 robot. The inverse kinematics were solved for using the forward kinematic model [4].

George, 2017 has proposed a robotic arm system with a set of improved technicalities, such as an improved wireless communication and soft catching gripper. The robotic arm can be controlled via android based smartphones or tablets. The robotic arm was implemented using Arduino Mega 2560 micro-controller, which enables Bluetooth based communication between the robotic arm and the user's phone [5].

III. Methodology

The journey it took to achieve the goal of receiving an object from our coupled team and place it in a desired position consisted of several steps, which are briefly explained in the subsequent subsections

A. Hardware Design & Adjustments

The used robotic arm model is the magnibot, which we found on grabCAD, however, we made a change from the original model, where we removed one of the servo motors, specifically the servo responsible to rotate the end effector, since our application does not require us to perform any rotation for the end effector, figure 2 shows the SolidWorks design for the robot. Our robotic system was custom-designed and 3D-printed for precise joints, links, and end-effector manufacturing. This approach enabled rapid prototyping and a tailored fit for the robot's structure. Actuators, powered by an Arduino control unit, drive the joints smoothly, ensuring the system performs its tasks effectively. Figure 3 shows our robot (left) and the coupled team's robot (right) fixed on a common base, ready to perform the task scenario.

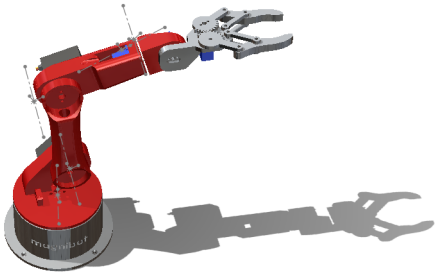

Fig. 2. The SolidWorks design

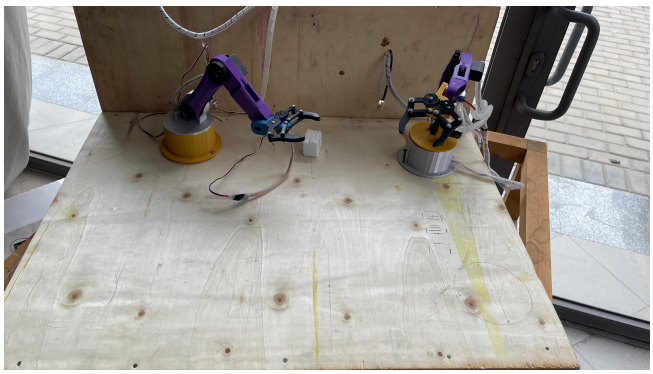

Fig. 3. The coupled robots

B. DH Convention

The Denavit-Hartenberg Convention table for our robotic manipulator was formed after a series of steps, were we first assigned the frames about all joints of interest, to get the

DH table, which we then used to compute the transformation matrix between every 2 frames, to have the total transformation matrix, figure 4 shows the assigned frames, and table 1 shows the DH table of our robot:

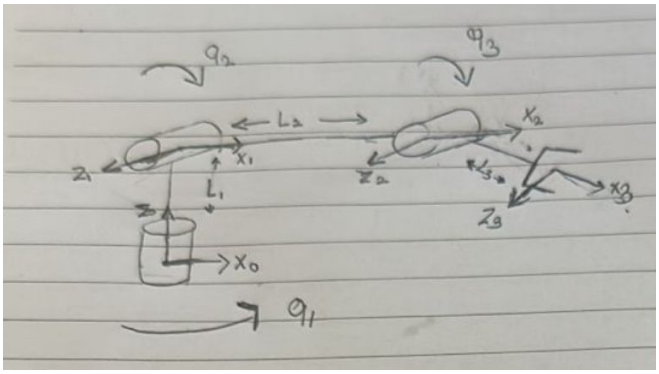

Fig. 4. Robot's assigned frames

i	θ_i	d_i	a_i	α_i
1	$\theta_1 = q_1$	L_1	0	$\frac{\pi}{2}$
2	$\theta_2 = q_2$	0	L_2	0
3	$\theta_3 = q_3$	0	L_3	0

TABLE I
DENAVIT-HARTENBERG PARAMETERS

C. MATLAB & Python functions

In order to validate the DH convention and the simulations, we implemented several functions to ensure that all simulations and codes provide accurate results. The forward position kinematics for a 3-joint robotic arm using the Denavit-Hartenberg (DH) convention. This method determines the end-effector's 3D position based on joint angles and link lengths. Joint angles (θ_{11}, θ_{12}, θ_{13}) are input in degrees and converted to radians for calculations. The robotic arm consists of three predefined link lengths: $l_1 = 4.5\,\text{cm}$, $l_2 = 12.1\,\text{cm}$, and $l_3 = 12.05\,\text{cm}$.

Using the DH parameters, transformation matrices T_1, T_2, and T_3 are computed for each joint, incorporating rotation, link offsets, and twists. The total transformation matrix, $T_{\text{total}} = T_1 \cdot T_2 \cdot T_3$, represents the cumulative effect of all transformations. The end-effector's position, denoted as $X = [X, Y, Z]^T$, is extracted from the first three rows of the fourth column of T_{total}, providing its 3D coordinates relative to the base frame.

Inverse kinematics (IK) determines the joint angles required to reach a desired position of the robotic arm's end-effector. The Newton-Raphson method, a numerical approach, is used to iteratively compute these angles. The target end-effector position (X_e, Y_e, Z_e) is provided as input, while the unknown joint angles $(\theta_1, \theta_2, \theta_3)$ are initialized with a guess of $[20°, 10°, 30°]$.

The forward kinematics function, based on the DH parameters, computes the current end-effector position, and an error function, $\mathbf{f}_{\text{vector}} = \begin{bmatrix} X - X_e \\ Y - Y_e \\ Z - Z_e \end{bmatrix}$, evaluates the difference from the target position. A Jacobian matrix J is computed symbolically to relate changes in joint angles to position changes. Using the Newton-Raphson update rule, $\Delta\mathbf{q} = -J^{-1}\mathbf{f}_{\text{vector}}$, the joint angles are iteratively adjusted.

The process repeats until the updates $\Delta\mathbf{q}$ are below a tolerance of 1×10^{-3} or a maximum of 500 iterations is reached. Upon convergence, the joint angles $[\theta_1, \theta_2, \theta_3]$ are returned as the solution. If convergence fails, a warning indicates that the target position may be unreachable or that a different initial guess may be required.

Forward velocity kinematics involves calculating the velocity of the end-effector based on joint velocities, using the Jacobian matrix to map joint velocities to both linear and angular end-effector velocities. The process begins by defining symbolic variables for the joint angles $(\theta_1, \theta_2, \theta_3)$, their derivatives $(\dot{\theta}_1, \dot{\theta}_2, \dot{\theta}_3)$, and the link lengths (l_1, l_2, l_3). Transformation matrices T_1, T_2, and T_3 are constructed using Denavit-Hartenberg (DH) parameters to represent the position and orientation of each link.

The rotational axes of the joints (J_{w1}, J_{w2}, J_{w3}) are derived from the z-axes of the transformation matrices. The position of the end-effector is computed using forward kinematics, and the positional Jacobian J_v is obtained by taking partial derivatives of the end-effector position $[f_x, f_y, f_z]$ with respect to each joint angle. The rotational Jacobian J_w, which represents angular velocity contributions, is formed using the rotational axes. The full Jacobian matrix J is constructed as:

$$J = \begin{bmatrix} J_v \\ J_w \end{bmatrix},$$

where J_v maps joint velocities to linear velocities, and J_w maps joint velocities to angular velocities.

The symbolic end-effector velocity V_{sym} is computed as:

$$V_{\text{sym}} = J_{\text{sym}} \cdot \begin{bmatrix} \dot{\theta}_1 \\ \dot{\theta}_2 \\ \dot{\theta}_3 \end{bmatrix}.$$

The Jacobian matrix is numerically evaluated at specified joint angles (q_1, q_2, q_3) and their velocities $(\dot{q}_1, \dot{q}_2, \dot{q}_3)$, resulting in the numeric velocity of the end-effector V.

Inverse velocity kinematics calculates the joint velocities $(\dot{q})$ required to achieve a desired end-effector velocity (V_{desired}). The Jacobian matrix J maps end-effector velocities to joint velocities, and its pseudoinverse is used to handle cases where J is not directly invertible.

The process begins by defining the joint angles $(\theta_1, \theta_2, \theta_3)$ and link lengths (l_1, l_2, l_3). Using Denavit-Hartenberg (DH) parameters, transformation matrices (T_1, T_2, T_3) are constructed to describe the position and orientation of each link. The positional Jacobian J_v is derived by differentiating the end-effector position with respect to the joint angles, while the rotational Jacobian J_w is formed from the z-axes of the transformation matrices.

The full Jacobian matrix J is given by:

$$J = \begin{bmatrix} J_v \\ J_w \end{bmatrix},$$

where J_v maps joint velocities to linear velocities, and J_w maps joint velocities to angular velocities. The symbolic Jacobian is evaluated at the specified joint angles (q_1, q_2, q_3) to obtain the numerical Jacobian J_{num}.

Since J_{num} may not be invertible, the pseudoinverse is computed as:

$$\text{Pseudo}J = \left(J_{\text{num}}^T J_{\text{num}}\right)^{-1} J_{\text{num}}^T.$$

The required joint velocities $\dot{q}$ are then calculated as:

$$\dot{q} = \text{Pseudo}J \cdot V_{\text{desired}}.$$

This provides the joint velocities needed to achieve the specified linear and angular velocities of the end-effector.

After completing the position and velocity kinematics, the trajectory of our desired application was studied, based on the robot's dimensions and the expected positions and available work area, using the inverse kinematics function previously implemented, the chosen trajectory type was joint space, due to its ease on the hardware and smoothness in movement, when compared to the task space, which forces the end effector to follow a series of points. Our trajectory is separated into 3 trajectories, the first trajectory focuses on moving the robotic arm from its starting position to the position of the object, an adjusting small halt is added, to refine the position and ensure it has reached to the object, and to give time for the gripper's servo to close and clamp to the object, then the second trajectory raises the object from the ground and moves it to a new position, and the third trajectory lowers the object to its new position, and the gripper is opened to release the object.

D. Simulations

1) Simscape: In order to simulate the robot on simscape, we exported the solidworks model to simulink via simscape, where we assigned the frames to match that of the DH convention, figure 5 shows the robot's zero position after assigning the frames, which was done by applying rigid transform before and after the joint blocks, and rotating the frames till they are aligned. An offset was later added to the input angles to ensure that the robot's zero position on simscape is the same as the zero position in the hardware and in CoppeliaSim. To validate the position kinematics, the input

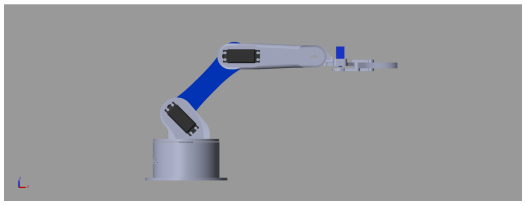

Fig. 5. Robot's zero position after frames assignment

to the joints was angles block, when assessing the velocity kinematics, the input was changed to $\dot{q}_1, \dot{q}_2, \dot{q}_3$ entering to an integrator block.

2) CoppeliaSim:

IV. Results

A. Simulation Results

1) Simscape: We first validated the forward position kinematics method, with a test case of $q_1 = 30, q_2 = 55$ & $q_3 = 81$, figure 6 shows the end effector positions from the matlab function, and figure 7 shows the positions from the simscape simulation, it can be seen that the error lies within 1cm, which is an acceptable range. We then tested the velocity

```
End effector position:
X = -7.12
Y = -4.11
Z = 28.46
```

Fig. 6. The end effector positions

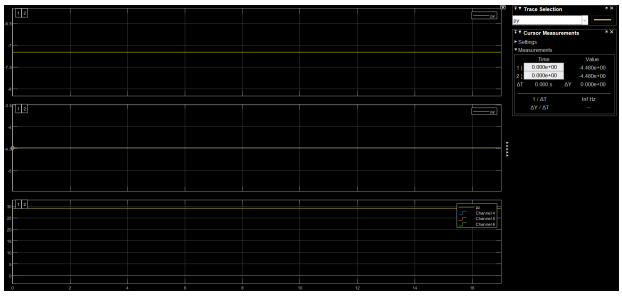

Fig. 7. Simscape position plots

to ensure that they align with each other, figures 8 and 9 show the values after simulation, at values $\dot{q}_1 = 0.1, \dot{q}_2 = 0.2$ & $\dot{q}_3 = 0.3$, at $t = 5s$, both the values from the code and the simscape simulations align in values, which confirms that the code runs properly and ready to be used in trajectory planning.

```
Numeric End-Effector Velocity (V):
   -6.7913
   -4.8884
   -7.1153
    0.2397
   -0.4388
    0.1000
```

Fig. 8. Velocity values at t = 5s

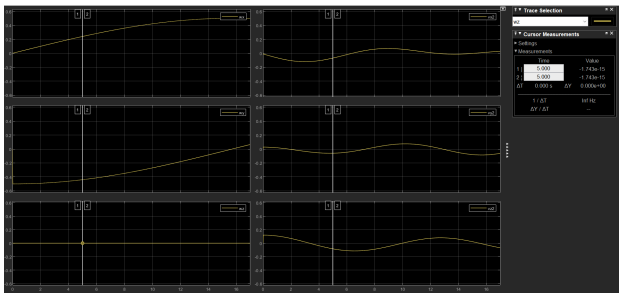

Fig. 9. Velocity plots

2) CoppeliaSim:

B. Hardware Results

After completing the position and velocity kinematics, a joint space trajectory was then implemented and validated on Simscape and CoppeliaSim, before being deployed to the hardware via a lookup table from the angles gathered from the trajectoey MATLAB codes, our trajectory is seperated into 3 trajectories, the first trajectory focuses on moving the robotic arm from its starting position to the position of the object, an adjusting small halt is added, to refine the position and ensure it has reached to the object, and to give time for the gripper's servo to close and clamp to the object, then the second trajectory raises the object from the ground and moves it to a new position, and the third trajectory lowers the object to its new position, and the gripper is opened to release the object. The video submitted in the repository validates this functionality.

V. Conclusion & Future Work

References

[1] R. Prakash, L. Behera, S. Mohan, and S. Jagannathan, "Dual-loop optimal control of a robot manipulator and its application in warehouse automation," *IEEE Transactions on Automation Science and Engineering*, vol. 19, no. 1, pp. 262–279, 2020.

[2] C. Martinez, R. Boca, B. Zhang, H. Chen, and S. Nidamarthi, "Automated bin picking system for randomly located industrial parts," in *2015 IEEE International Conference on Technologies for Practical Robot Applications (TePRA)*, pp. 1–6, 2015.

[3] V. Borkar and G. Andurkar, "Development of pick and place robot for industrial applications," *International Research Journal of Engineering and Technology*, vol. 4, no. 09, 2017.

[4] S. Kumar, A. Majumder, S. Dutta, R. Raja, S. Jotawar, A. Kumar, M. Soni, V. Raju, O. Kundu, E. H. L. Behera, *et al.*, "Design and development of an automated robotic pick & stow system for an e-commerce warehouse," *arXiv preprint arXiv:1703.02340*, 2017.

[5] A. Baby, C. Augustine, C. Thampi, M. George, A. AP, and P. C. Jose, "Pick and place robotic arm implementation using arduino," *IOSR Journal of Electrical and Electronics Engineering*, vol. 12, no. 2, pp. 38–41, 2017.